FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Jose Miguel Viado
Section:	AN42
Professor:	Ms. Crisola Tan
Date:	10/05/26

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

RBR was easier to implement because it only uses simple if then rules, so it was easier to understand and code. CBR is more flexible for unstructured problems because it can look at past cases and use a similar solution. RBR is better for situations where the data keeps changing because the rules can check the new information right away. Some real-world examples of RBR are smart home systems and medical diagnosis, while CBR can be used for customer support and troubleshooting.



Your Code - Laboratory Tasks

```
facts = {
    "temperature": 32,
    "raining": False,
    "humidity": 80,
    "person_present": True,
    "aircon_on": False
}

def rule1(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"

def rule2(facts):
    if facts["humidity"] > 70:
        return "Turn on the fan"

def rule3(facts):
    if facts["person_present"] and not facts["raining"]:
        return "Open the window"

rules = [rule1, rule2, rule3]

actions = []

for rule in rules:
    result = rule(facts)
    if result:
        actions.append(result)

print("Rule-Based Reasoning Actions:")
for action in actions:
    print("-", action)

case_base = [
    {
        "problem": {"temperature": 32, "raining": False},
        "solution": "Turn on the aircon"
    },
    {
        "problem": {"temperature": 25, "raining": True},
        "solution": "Keep the windows closed"
    },
```

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Turn on the fan
- Open the window

CBR Reused Solution:
Turn on the aircon
Change the solution? (yes/no): yes
Enter new solution: Open the window
New case added to the case base.
```

```
    {
        "problem": {"temperature": 30, "raining": False},
        "solution": "Open the window"
    }
]


new_problem = {
    "temperature": 32,
    "raining": False
}

best_case = None
best_score = 0

for case in case_base:
    score = 0

    for key in new_problem:
        if new_problem[key] == case["problem"][key]:
            score += 1

    if score > best_score:
        best_score = score
        best_case = case

solution = best_case["solution"]

print("\nCBR Reused Solution:")
print(solution)

change = input("Change the solution? (yes/no): ")

if change == "yes":
    solution = input("Enter new solution: ")

case_base.append({
    "problem": new_problem,
    "solution": solution
})

print("New case added to the case base.")
```